

American University of Beirut • Maroun Semaan Faculty of Engineering and Architecture • Department of Mechanical Engineering

Amphibious PrandtlPlane UAV

Flight Control Team Onboarding

Project goals, design workflow, and the plan for autonomous water landing

Supervisor Dr. Bilal Kaddouh

Team Jamil Chahine (lead, control), Zahi (control, simulation), Tohme (robotics onboarding)

Program VIP Research, Fall 2026

Version 1.0 — 22 September 2026

Read this first

This document is the single reference for anyone joining the flight-control team. It explains what the aircraft is, what has already been done, what we are building this semester, and *how* we are going to build it. The method matters as much as the result: we design the controller on paper and in MATLAB against explicit specifications, verify it there, and only then fly it in the simulator. The simulator is where we confirm the design, not where we discover it. If you read only one section, read Section 4.

Contents

1	The project in one page	1
1.1	Why the box wing matters for control	2
2	What has been done so far	2
3	Goals for this semester	3
4	How we work: the design workflow	4
4.1	One source of truth	4
5	The mathematics, stage by stage	4
5.1	Requirements and the touchdown envelope	4
5.2	The plant: rigid-body 6-DOF model	5
5.3	Trim and linearisation	5
5.4	Controller architecture	6
5.5	Specifications every loop must meet	7
5.6	Mapping the design onto PX4	7
5.7	The water landing law	7
6	The simulator	8
6.1	Why PX4 and Gazebo Harmonic	8
6.2	Setting up the water without repeating March	9

7	Verification and validation campaign	9
8	Team, roles and timeline	10
8.1	Who does what	10
8.2	Tohme’s onboarding path	10
8.3	Twelve-week schedule	11
8.4	Risks	11
8.5	Decisions we need from the supervisor	11
9	In plain terms	11
10	References and links	12

1 The project in one page

The parent project asks whether a fleet of small autonomous amphibious aircraft could complement or replace the Bombardier CL-415 in aerial firefighting. The CL-415 scoops roughly 6 000 L of water in twelve seconds of surface skimming and delivers it in a single drop. It is effective, but expensive and limited to large bodies of water. A swarm of small amphibians operating from lakes, reservoirs and rivers is an attractive alternative, but only if each vehicle can land on water, scoop, take off and drop its load autonomously and reliably.

Our aircraft is a 1:11 subscale **PrandtlPlane**: a closed box-wing configuration in which a front wing and a rear wing are joined at the tips by vertical fins. It carries a 4 kg water payload, about 29% of its take-off mass, which mirrors the mass fraction of the full-size CL-415. The conceptual sizing was done in the parent thesis and is summarised in Table 1. Those numbers are the design point; treat them as authoritative unless the supervisor changes them.

Table 1: Aircraft design point (from the conceptual sizing tool). This is the reference for every model we build.

Parameter	Value	Note
MTOW (loaded)	13.96 kg	rounded to 14 kg in analysis
Empty mass (post-drop)	9.96 kg	rounded to 10 kg
Water payload	4.00 kg	28.7% of MTOW, tank 0.12 m below CG
Wingspan b	2.89 m	standard geometry ; the 2.0 m UE5 mesh is retired
Reference area S	0.930 m ²	mean chord 0.322 m, AR 9.0, $e = 0.68$
Cruise speed	20 m/s	design drop condition
Stall speed V_s	13.2 m/s	clean, loaded; about 11.2 m/s empty
$C_{L,\max}$	1.37	
Drag polar	$C_D = 0.055 + 0.052 C_L^2$	$L/D = 9.35$ at $C_L = 1.03$
Propulsion	2×479 W tractor	cruise thrust 16.7 N, $T/W = 0.245$, thrust line through loaded CG
Loaded trim (20 m/s)	$\alpha = 6.33^\circ$, $\delta_e = -1.83^\circ$	throttle 27.9%
Unloaded trim (20 m/s)	$\alpha = 4.46^\circ$, $\delta_e = -0.59^\circ$	throttle 24.4%

1.1 Why the box wing matters for control

A conventional aircraft pitches with a tail. A PrandtlPlane pitches by working its two wings against each other: deflecting the front-wing surfaces one way and the rear-wing surfaces the other way produces a pure pitching couple, while deflecting them the *same* way changes lift with almost no

pitch change. The second mode is called direct lift control (DLC) and it is a genuine advantage for a smooth water arrival, because it lets us adjust sink rate without disturbing the touchdown attitude. Figure 1 shows the idea. The price is that we cannot borrow stability derivatives from a conventional aircraft; the two wings interfere with each other aerodynamically and the pitch damping is unusually high. The Pisa group that built and flew IDINTOS, an amphibious light PrandtlPlane, reported that stall begins on the front wing and that the rear-wing surfaces stay effective through it, which is exactly the behaviour we want on a slow approach.

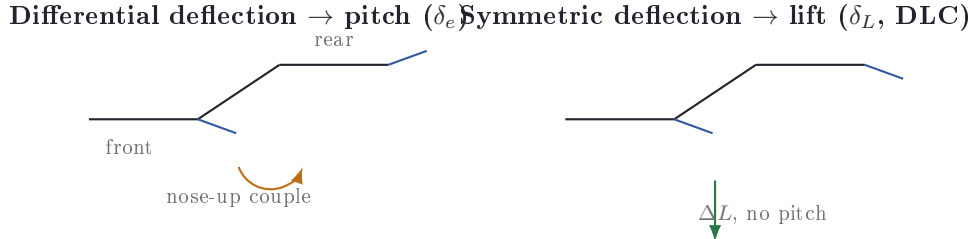


Figure 1: Control mixing on the PrandtlPlane (side view, flow from the left). The autopilot commands δ_e and δ_L ; a fixed mixing matrix turns them into front and rear surface deflections. AVL tells us how cleanly the two modes separate.

2 What has been done so far

The project has been through four phases. Knowing them saves you from repeating them.

Gazebo + ArduPilot (March 2026). A first single-vehicle simulator was built in Gazebo Harmonic with ArduPlane SITL, using the LiftDrag plugin for aero and the VRX buoyancy and hydrodynamics plugins for water. The hull floated only once its mesh collision was replaced by box primitives; mesh buoyancy does not work, and the ODE physics engine crashed when more than two collision primitives shared a link. Both facts shape how we set up water in Section 6.

Unreal Engine 5 + Cosys-AirSim + PX4 (April–May 2026). For photorealistic rendering and future reinforcement-learning training the simulator moved to UE5 with a custom water layer. This is the long-term high-fidelity environment. It is *not* where we design the controller.

Longitudinal drop study in MATLAB (April–May 2026). The supervisor asked how much the controller has to do when 4 kg of water leaves the aircraft in flight. Two LQR-with-integral controllers were designed for the loaded and unloaded plants with identical weights and cross-tested; four gain-scheduling strategies were compared; and a nonlinear 6-DOF simulation of the drop was run. The headline results are in Table 2. The recommendation is an instantaneous gain switch triggered by the drop-bay flag, with no mass estimator in the loop. The main caveats are that the pitch derivatives were handbook estimates ($C_{m_\alpha} = -0.80$, $C_{m_q} = -12$, $C_{m_{\delta_e}} = -1.20$), the inertia tensor had no off-diagonal terms, and the lateral-directional axes were not studied at all.

Table 2: Drop study, step release of 4 kg at 20 m/s cruise. Any sensible controller turns a 30 m balloon into a 20 cm event.

Case	Peak Δh	Peak θ	Peak δ_e	Settle
Open loop	+30.2 m	18.2°	1.8°	never
Loaded gains kept	+0.59 m	2.6°	1.8°	2.2 s
Instant switch on drop flag	+0.16 m	2.4°	2.5°	< 0.5 s
1 s linear blend	+0.17 m	2.4°	1.8°	< 0.5 s

The efficiency and fleet comparison that started the project is now out of scope. The project is a control and simulation project for one aircraft.

3 Goals for this semester

The near-term objective is a properly designed flight controller for our aircraft, validated in a lower-fidelity simulator, that flies the full air mission and lands on water smoothly and autonomously. Planing, scooping and water take-off come afterwards. Because there are three of us, the goals are tiered so that we can over-deliver without losing the core.

Tier 1 — must deliver

A validated nonlinear 6-DOF model built from an AVL box-wing analysis, trimmed and linearised from approach speed to cruise at both 14 kg and 10 kg. A cascaded PID autopilot designed in MATLAB against written specifications, ported to PX4, flying our aircraft in Gazebo through take-off, climb, cruise, loiter and waypoint tracking. An autonomous water landing that arrives inside a defined touchdown envelope, validated over roughly one hundred randomised runs.

Tier 2 — should deliver

An LQR/LQI design on the same model, compared head-to-head with the PID on tracking, robustness and control effort. The drop-flag gain switch integrated into the flight controller, so the same autopilot survives the 4 kg release. A custom flare law if PX4's stock landing logic cannot hold the touchdown attitude window.

Tier 3 — stretch (potentially publishable)

A direct-lift-control flare that holds pitch constant and trims sink rate with symmetric wing deflection. A landing-in-waves envelope sweep. Hardware-in-the-loop on a real Pixhawk before any flight hardware exists.

4 How we work: the design workflow

Every task in this project belongs to one stage of the pipeline in Figure 2. Each stage has inputs, outputs and an acceptance criterion, and we do not move to the next stage until the current one passes. This is the difference between a controller we can defend and a set of gains that happened to work once.

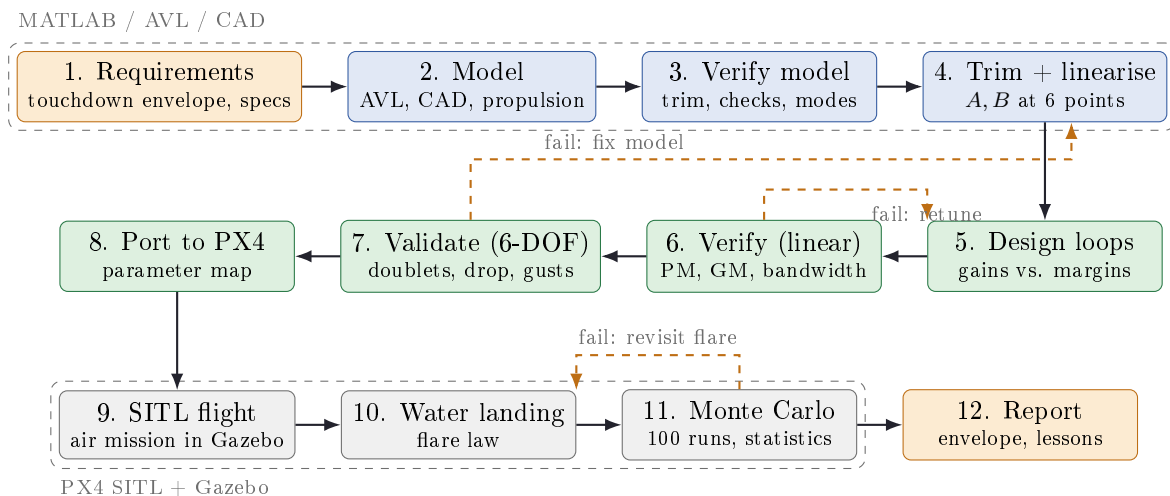


Figure 2: The design pipeline. Blue and green stages live in MATLAB; grey stages live in the simulator. The dashed arrows are the only ways backwards, and each one names what failed.

4.1 One source of truth

All aircraft numbers (geometry, mass and inertia, derivatives at each trim point, actuator limits, thrust curve) live in a single `params.yaml`. The MATLAB model reads it, and the Gazebo SDF and plugin parameters are *generated* from it by a script. Nobody types a number into an SDF by hand. This is the only way to guarantee that the model we designed against and the model we fly in the simulator are the same aircraft.

5 The mathematics, stage by stage

This section is the technical core. It is written so that a third-year student can follow it with Beard and McLain open beside it; chapter references are given where they help.

5.1 Requirements and the touchdown envelope

Cruise and loiter requirements are conventional: hold 20 m/s and altitude within about ± 1 m/s and a few metres in still air, and fly coordinated turns up to 30° bank. The approach is flown at $1.3 V_s$, about 17 m/s loaded or 15 m/s empty (the aircraft normally lands empty, before scooping), on a -3° to -4° glide slope. The landing requirements come from seaplane practice, and in particular from the glassy-water technique in the FAA Seaplane Handbook (FAA-H-8083-23): over calm water a pilot cannot judge height, so the procedure is to establish the touchdown attitude and a low, steady sink rate, and then simply wait for contact without trying to flare. Our autopilot faces exactly this problem, because rangefinders are unreliable over a mirror-flat surface. The proposed envelope, to be confirmed with the supervisor, is in Table 3.

Table 3: Proposed touchdown envelope. Every Monte Carlo run is scored against these lines.

Quantity	Limit	Why
Sink rate at contact	≤ 0.75 m/s (fail above 1.5)	seaplane target of 100–200 ft/min; hull loads
Pitch at contact	planing trim to $+8^\circ$	step touches first; bow never digs in; not tail-first
Roll at contact	$\leq 3^\circ$, no side drift	a wingtip or sponson catching the water is the classic seaplane accident
Airspeed at contact	1.05 to $1.2 V_s$	above stall, below the speed at which the hull skips
Porpoising	pitch oscillation decays within 2 cycles	unstable planing trim grows without limit
Runout	no nose-over, stops within a set distance	throttle cut and elevator held up after contact

5.2 The plant: rigid-body 6-DOF model

The aircraft is a rigid body with twelve states and four control inputs,

$$\mathbf{x} = [u \ v \ w \ p \ q \ r \ \phi \ \theta \ \psi \ p_N \ p_E \ h]^T, \quad \boldsymbol{\delta} = [\delta_e \ \delta_a \ \delta_r \ \delta_t]^T, \quad (1)$$

with the body-axis equations of motion (Beard and McLain, Chapter 3)

$$\dot{\mathbf{x}} = f(\mathbf{x}, \boldsymbol{\delta}) = \text{kinematics} + \frac{1}{m}(\mathbf{F}_{\text{aero}} + \mathbf{F}_{\text{thrust}} + \mathbf{F}_{\text{grav}} + \mathbf{F}_{\text{hydro}}), \quad (2)$$

and the corresponding moment equations using the full inertia tensor, including I_{xz} once the SolidWorks model delivers it. $\mathbf{F}_{\text{hydro}}$ is zero in the air and switches on at water contact, so one model covers flight and touchdown. In the code we use a quaternion for attitude to avoid singularities near the nose-up landing attitude, and Euler angles only for plotting.

The aerodynamic forces use the stability-derivative form (Beard and McLain, Chapter 4):

$$C_L = C_{L_0} + C_{L_\alpha} \alpha + C_{L_q} \frac{q\bar{c}}{2V} + C_{L_{\delta_e}} \delta_e + C_{L_{\delta_L}} \delta_L, \quad C_D = C_{D_0} + KC_L^2, \quad (3)$$

$$C_m = C_{m_0} + C_{m_\alpha} \alpha + C_{m_q} \frac{q\bar{c}}{2V} + C_{m_{\delta_e}} \delta_e, \quad (4)$$

and the same structure for C_Y , C_l and C_n in terms of β , p , r , δ_a and δ_r . The polar constants $C_{D_0} = 0.055$ and $K = 0.052$ come from the sizing tool. Every other derivative comes from AVL, replacing the handbook values used in the drop study. The propulsion model is a thrust map $T(V, \delta_t)$ built from APC propeller data and the motor constants, anchored to 16.7 N at cruise trim, and it keeps the thrust-line geometry so that the throttle-to-pitch coupling after the drop appears naturally. Servos are first-order lags with $\tau = 0.05$ s plus rate and deflection limits; those limits size the inner-loop bandwidth later.

Model verification before anything else

At each trim point lift must equal weight and every moment must vanish. Remove all forces and check that energy and momentum are conserved. Compare the modes of the linearised model with the ones AVL predicts and with handbook ranges for a 14 kg, AR 9 aircraft. Apply a small elevator doublet and confirm a decaying short-period of sensible frequency. A model that fails any of these is fixed before a single gain is computed.

5.3 Trim and linearisation

Trim is the solution of $f(\mathbf{x}^*, \boldsymbol{\delta}^*) = 0$ for steady flight at a chosen airspeed V and flight-path angle γ , found with `fsolve` or Simulink's `findop`. We trim at six points: cruise (20 m/s, level), approach ($1.3 V_s$ on a -4° glide) and slow flight near stall, each at 14 kg and 10 kg. The cruise trims must reproduce the values in Table 1. Linearisation is the numerical Jacobian at each trim,

$$\dot{\tilde{\mathbf{x}}} = A \tilde{\mathbf{x}} + B \tilde{\boldsymbol{\delta}}, \quad A = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_*, \quad B = \left. \frac{\partial f}{\partial \boldsymbol{\delta}} \right|_*, \quad (5)$$

which decouples into a longitudinal set $\tilde{\mathbf{x}}_{\text{lon}} = [u \ w \ q \ \theta \ h]$ with inputs $[\delta_e \ \delta_t]$ and a lateral-directional set $\tilde{\mathbf{x}}_{\text{lat}} = [v \ p \ r \ \phi \ \psi]$ with inputs $[\delta_a \ \delta_r]$ (Beard and McLain, Chapter 5). The eigenvalues give the five classical modes. Table 4 lists the targets we compare against; they are the Level 1 flying-qualities bands from MIL-F-8785C, used here as design guidance rather than certification rules. Expect the short period to be very well damped on a box wing. The lateral-directional set is where box wings are known to be marginal, so scrutinise dutch roll and spiral.

Table 4: Open-loop mode targets (MIL-F-8785C Level 1, Class I aircraft).

Mode	States	Target
Short period	w, q	$0.35 \leq \zeta_{sp} \leq 1.3$, CAP in band
Phugoid	u, θ	$\zeta_{ph} \geq 0.04$
Roll subsidence	p	$\tau_r \leq 1.4$ s
Spiral	ϕ, ψ	time to double ≥ 20 s
Dutch roll	v, r	$\zeta_d \geq 0.08$, $\omega_d \geq 0.4$ rad/s, $\zeta_d \omega_d \geq 0.15$

5.4 Controller architecture

We use a cascaded architecture built by successive loop closure (Beard and McLain, Chapter 6). It is chosen because it is exactly the structure of PX4's fixed-wing stack, so every gain we compute has a named PX4 parameter, and because each loop can be designed and verified on its own. Figure 3 shows it.

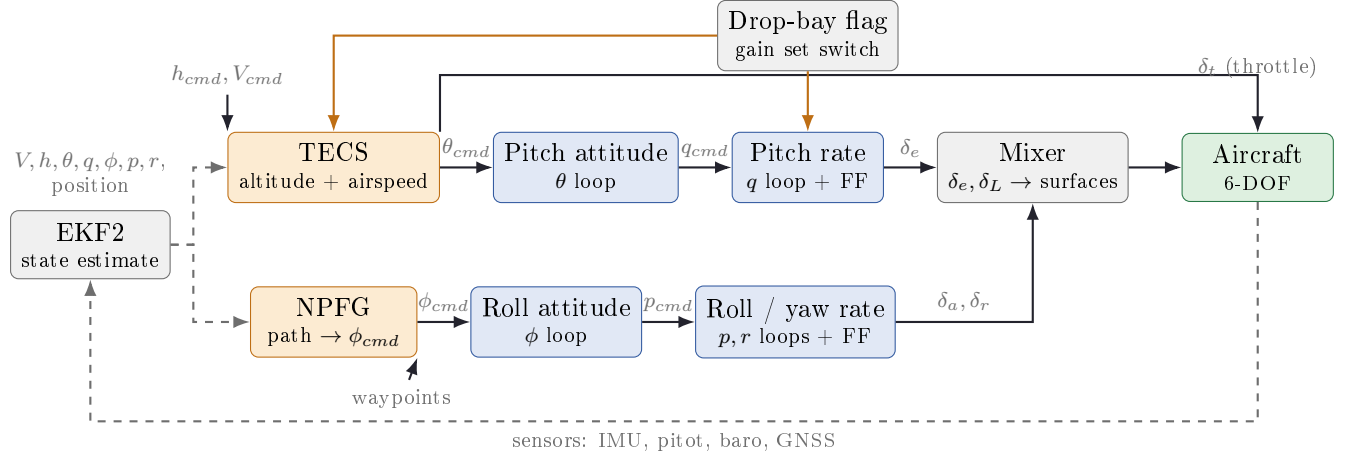


Figure 3: Cascaded autopilot. Inner rate loops with feedforward, attitude loops around them, TECS for the energy axis and NPFG for lateral guidance. Moving inward, each loop is five to ten times faster than the one outside it. The drop-bay flag switches the throttle-pitch coupling gains, reusing the drop-study result.

Inner rate loops. The fastest loops, one per axis, are proportional-integral with a large feedforward term. Following Beard and McLain, the proportional gain is bounded by actuator authority: with a maximum expected error $e_{\max}$ and a surface limit $\delta_{\max}$, the design starts at $k_p \approx \delta_{\max}/e_{\max}$ and is then trimmed to hit a chosen damping ratio. The feedforward is the steady surface deflection needed to hold a commanded rate, which the linear model gives directly.

Attitude loops. Roll and pitch angle loops close around the rate loops. They must be five to ten times slower so that the inner loop looks like a unit gain to them; if they are not, the aircraft develops a slow oscillation. PX4 exposes this as a time constant per axis.

TECS. Altitude and airspeed are coupled through the aircraft’s energy. Define the specific total energy and its distribution,

$$E_T = gh + \frac{1}{2}V^2, \quad \dot{E}_T \approx gV \left(\gamma + \frac{\dot{V}}{g} \right), \quad \dot{E}_B \approx gV \left(\gamma - \frac{\dot{V}}{g} \right). \quad (6)$$

Throttle controls the total-energy rate $\dot{E}_T$ and pitch controls the balance $\dot{E}_B$ between potential and kinetic energy. TECS sits on top of the pitch loop, so the pitch loop must be tuned first.

Lateral guidance. A heading command becomes a bank command through the coordinated-turn relation $\dot{\psi} = g \tan \phi / V$; PX4’s NPFG path-following supplies ϕ_{cmd} from the cross-track error.

5.5 Specifications every loop must meet

Each loop is verified on the linear model before it is closed in the nonlinear one. The pass criteria are: phase margin at least 45° , gain margin at least 6 dB, bandwidth separation of at least $5\times$ between successive loops, no actuator saturation for the expected command steps, and all of the above still true when C_{m_α} and C_{m_q} are varied by $\pm 30\%$ and the mass by $\pm 10\%$. An LQR/LQI design with Bryson’s rule for Q and R is carried alongside as the analytical benchmark, and it is the natural home for the drop-flag gain switch from the earlier study.

5.6 Mapping the design onto PX4

Because the architecture is the same, porting is a table lookup (Table 5). PX4’s fixed-wing autotune, which injects sine sweeps and identifies the rate dynamics online, is used afterwards as an independent check: if autotune proposes gains close to ours, the model is confirmed; if they differ by more than a factor of two, there is a modelling error to find.

Table 5: Where each designed gain goes in PX4.

Design quantity	PX4 parameters
Rate-loop k_p, k_i , feedforward	FW_RR_P, FW_RR_I, FW_RR_FF (roll); FW_PR_* (pitch); FW_YR_* (yaw)
Attitude-loop time constants	FW_R_TC, FW_P_TC
TECS energy loops	FW_T_CLMB_MAX, FW_T_SINK_MIN, FW_T_SINK_MAX, FW_T_* time constants and energy weights
Lateral guidance	NPGF_* period and damping
Approach and flare	FW_LND_ANG, FW_LND_AIRSPD, FW_LND_FLALT, FW_LND_FL_TIME, FW_LND_FL_PMIN, FW_LND_FL_PMAX, FW_LND_TD_DIST
Landed detection on water	LNDFW_* thresholds (airspeed, velocity, rotation)

5.7 The water landing law

On approach the aircraft tracks the glide slope with TECS. Below a flare height h_f the commanded sink rate shrinks with height,

$$\dot{h}_{cmd} = -\frac{h}{\tau} - \dot{h}_{td}, \quad h_f = \tau(|\dot{h}_{app}| - \dot{h}_{td}), \quad (7)$$

where τ is the flare time constant and $\dot{h}_{td}$ the residual sink rate at contact. At 15 m/s on a -4° glide the approach sink rate is about 1.05 m/s; with $\tau = 1$ s and a 0.3 m/s target the flare begins around 0.75 m. Figure 4 shows the profile.

The catch is that this law needs height, and over glassy water height is exactly what we cannot trust. So the primary strategy mirrors the seaplane pilot: fly the last segment at a fixed touchdown attitude and a low, fixed sink rate, hold it, and let contact happen. Barometric and GNSS altitude relative to the known water plane at $z = 0$ gives the coarse height needed to set up the segment; nothing in the last metre depends on a rangefinder. We will first try to configure PX4’s own FW_LND_* landing with the water surface as ground level, and we will implement the constant-attitude arrival as a small custom module or a ROS 2 offboard controller. The PrandtlPlane’s direct-lift channel is the stretch version of the same idea: pitch is held constant by δ_e while δ_L trims the sink rate. NASA’s Twin Otter and the Lockheed L-1011 both used direct lift for exactly this kind of constant-attitude flare. After land detection the throttle is cut, the elevator is held up so the bow stays high, and the rudder holds heading until the aircraft stops.

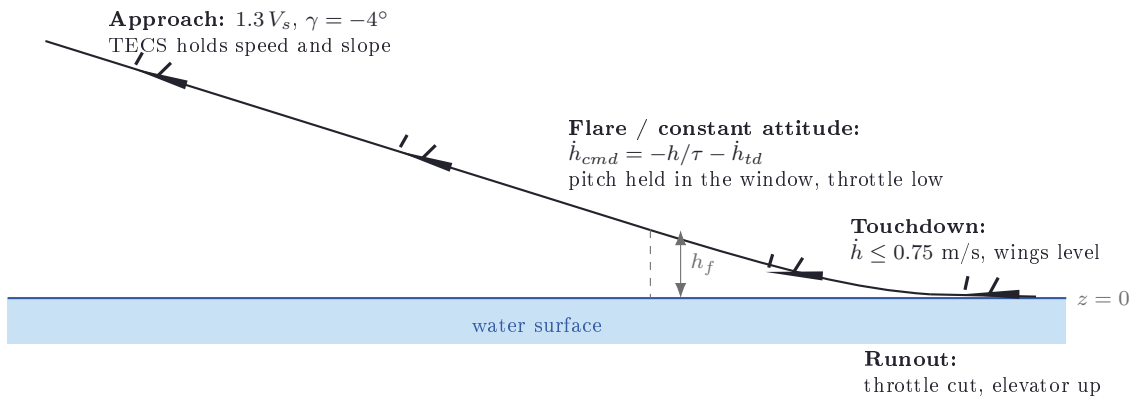


Figure 4: Water landing profile. The flare law is used when a height reference is available; over glassy water the last segment is flown at a fixed attitude and sink rate, and the aircraft waits for contact.

6 The simulator

6.1 Why PX4 and Gazebo Harmonic

The controller’s deployment target is PX4, so the simulator must run PX4’s actual flight code, and Gazebo Harmonic is PX4’s supported physics back end. Two features make it the right choice for us. The **AdvancedLiftDrag** plugin was written to take AVL-style derivatives directly, and PX4 ships an **avl_automation** tool that turns an AVL run into plugin parameters. That means the MATLAB model and the Gazebo model come from the same AVL file. Gazebo also has first-class buoyancy and hydrodynamics systems, and the maritime community (VRX, **asv_wave_sim**) has already solved floating hulls and wave fields. The stack is in Figure 5.

JSBSim is a stronger flight-dynamics engine on its own and remains a useful cross-check for the air phase, but it has no water. FlightGear and RealFlight are for piloted feel, not scripted campaigns. Isaac Sim, Pegasus and the existing UE5 Cosys-AirSim environment are the high-fidelity phase, deferred until the controller is proven.

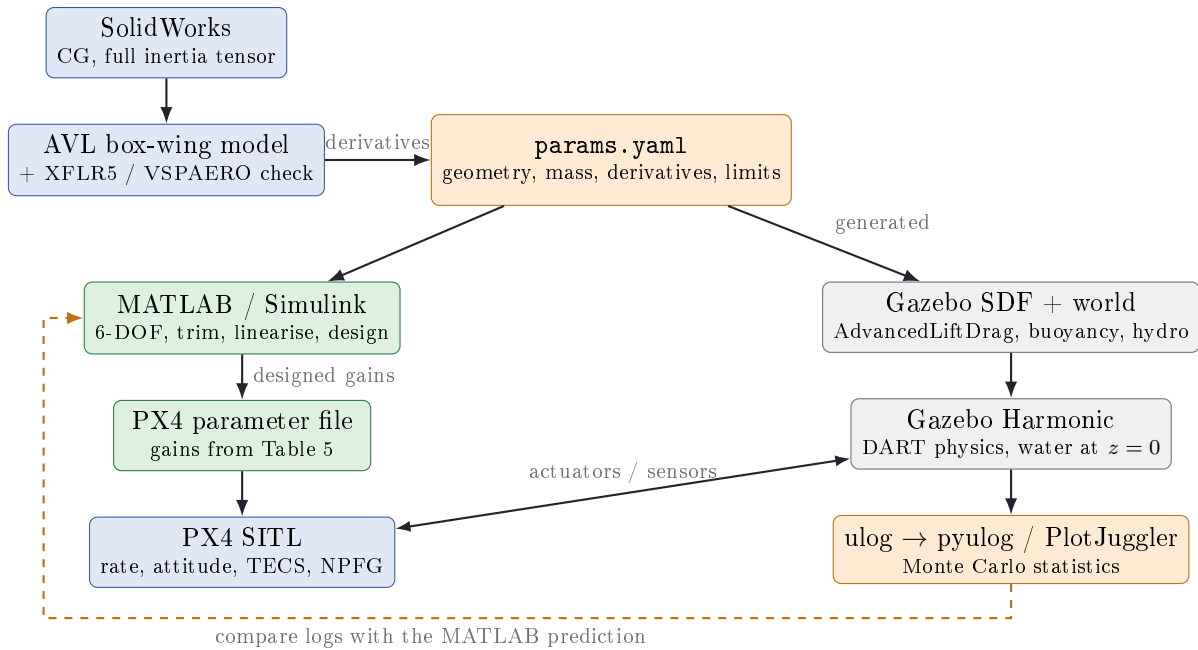


Figure 5: Tool chain. Everything downstream is generated from one parameter file, and the simulator’s logs are compared with the MATLAB prediction rather than tuned on their own.

6.2 Setting up the water without repeating March

The March prototype taught us three things, and the recipe below is built on them. Use the **DART** physics engine rather than ODE; it is more stable with constraints and it integrates added mass implicitly through the SDF **fluid_added_mass** tag, which is far better behaved for a light floating body than an explicit added-mass force. Represent the hull as **one link with a few box collision volumes** along its length, not a mesh and not many primitives spread over jointed links; several boxes are needed so that buoyancy produces pitch and roll moments rather than a single upward force. Use the Gazebo **Buoyancy** system in graded mode and the **Hydrodynamics** system for Fossen-style linear and quadratic damping, or the VRX and **asv_wave_sim** **Surface** plugin, which samples points on the hull and later gives us waves for free. The water is a plane at $z = 0$ with no collision; it acts only through forces. Planing lift, which grows with speed and trim angle, has no stock plugin; for landing we do not need it, and when we do it will be a small custom plugin implementing the Savitsky method.

Reuse before you build. PX4 ships working fixed-wing models (**gz_rc_cessna**, **gz_advanced_plane**) in the PX4-gazebo-models repository; the Gazebo “Create a surface vehicle” tutorial floats a WAM-V on a wave field; VRX and **asv_wave_sim** provide worlds, wave models and the Surface plugin;

and ArduPilot’s `ardupilot_gazebo` and the Intelligent-Quads `iq_sim` repositories have plane and boat examples to crib from. Links are collected in Section 10.

7 Verification and validation campaign

The test matrix in Table 6 runs from unit tests of the model to the Monte Carlo landing campaign. Every row has a pass criterion, and results are archived so that a change to the model can be re-verified in minutes. The Monte Carlo runs randomise steady wind and gusts, Dryden turbulence, sensor noise, mass within $\pm 10\%$ and the pitch derivatives within $\pm 30\%$, and score each run against Table 3. Logs are PX4 `.ulg` files, inspected interactively in Flight Review and PlotJuggler and processed in batch with `pyulog`. The repository is organised as `params/`, `matlab/`, `sim/`, `tests/` and `docs/`, with continuous integration running the unit and linear tests on every push.

Table 6: Test matrix. A row must pass before the rows below it are attempted.

Level	Tool	Test	Pass
Unit	MATLAB	aero build-up vs. AVL at fixed α, β	within tolerance
Unit	MATLAB	thrust curve vs. design point	16.7 N $\pm 5\%$ at cruise
Unit	MATLAB	trim solver vs. Table 1	α, δ_e , throttle match
Model	MATLAB	open-loop modes vs. Table 4	in band or explained
Linear	MATLAB	loop margins, all six trims	PM $\geq 45^\circ$, GM ≥ 6 dB
Nonlinear	Simulink	doublets, gusts, 4 kg drop	drop excursion ≤ 0.5 m, recovery ≤ 0.5 s
SITL	PX4 + Gazebo	cruise, loiter, approach tracking	speed and altitude errors in spec
SITL	PX4 + Gazebo	autotune cross-check	gains within $2\times$ of design
SITL	PX4 + Gazebo	nominal water landing	inside Table 3
Robustness	PX4 + Gazebo	100 randomised landings	$\geq 90\%$ inside envelope, no capsize
Stretch	Pixhawk HIL	repeat air mission	matches SITL

8 Team, roles and timeline

8.1 Who does what

Jamil leads requirements, the AVL longitudinal model, trim and linearisation, the longitudinal loops and TECS, and the flare and direct-lift law. **Zahi** owns the AVL lateral-directional model and mass properties, the nonlinear 6-DOF model, the lateral loops, the LQR/LQI benchmark and gain scheduling, the wind and gust models, and the simulator water setup. **Tohme** owns the infrastructure: the propulsion and actuator sub-models as a first self-contained task, the Gazebo SDF and world, porting gains to PX4 parameters, the Monte Carlo runner and the log pipeline. He also sits in on every controller design session; the theory is learned from real work, not in isolation.

8.2 Tohme’s onboarding path

The spine is Beard and McLain, *Small Unmanned Aircraft: Theory and Practice*, worked together with its companion Python simulator `byu-magicc/mavsim_public`. The book builds a fixed-wing autopilot in the same order as this project: Chapter 3 equations of motion, Chapter 4 forces and moments, Chapter 5 trim and linear models, Chapter 6 successive loop closure. Alongside it come the PX4 developer guide (SITL, Gazebo, fixed-wing tuning pages), the Gazebo maritime tutorials and ROS 2 basics. Each week produces one runnable artefact, as in Table 7.

Table 7: First four weeks for a new team member.

Week	Learn	Deliver
1	Install PX4, Gazebo Harmonic, QGroundControl (native Ubuntu 22.04/24.04 preferred). PX4 architecture: modules, uORB, parameters.	Fly <code>gz_rc_cessna</code> through take-off, loiter and auto-land; open the <code>.ulg</code> in PlotJuggler and plot airspeed, altitude, pitch.
2	Beard & McLain Ch. 3–4 with mavsims.	mavsims Ch. 4 running with our mass and geometry substituted in.
3	Beard & McLain Ch. 5; APC propeller data; servo models.	Propulsion thrust map $T(V, \delta_t)$ and actuator model in MATLAB, with unit tests.
4	Gazebo SDF, AdvancedLiftDrag, buoyancy tutorial.	Our SDF floating stably on the water world under DART.

8.3 Twelve-week schedule

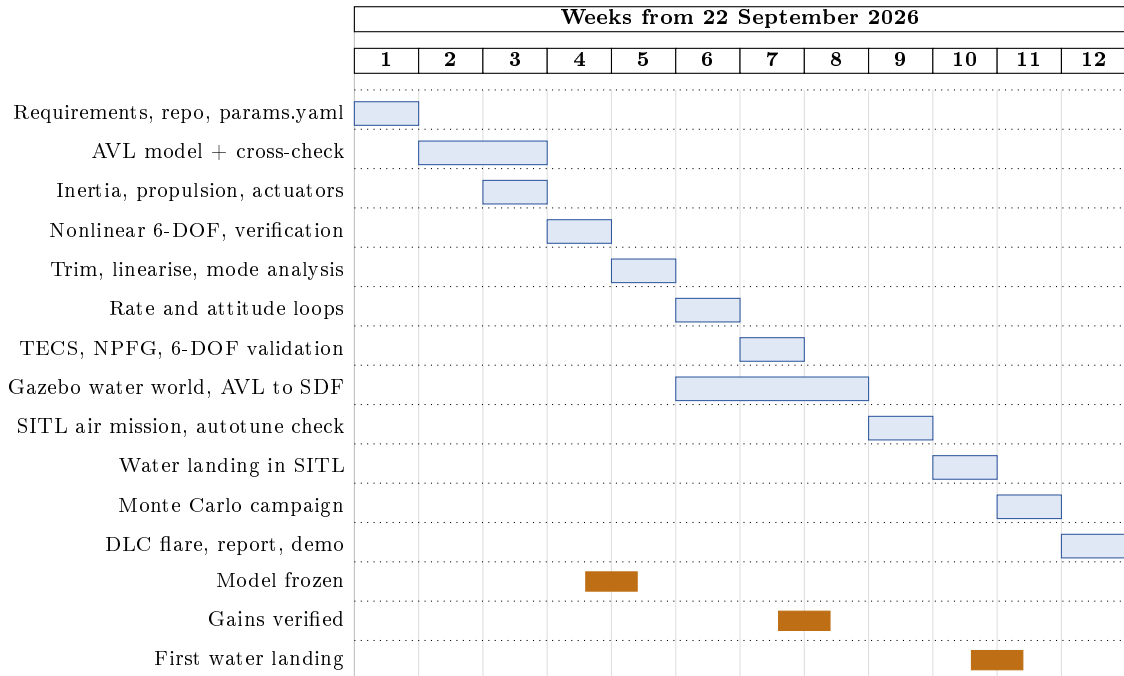


Figure 6: Schedule from the week of 22 September 2026. The simulator track (week 6 onward) runs in parallel with control design so that neither waits for the other.

8.4 Risks

The SolidWorks model slips. Proceed with the mass-station inertia estimate and $I_{xz} = 0$, flagged, and re-run linearisation when CAD arrives. **Lateral-directional derivatives disagree between tools.** Carry the spread into the uncertainty sweeps and design conservatively; the literature says classical methods over-predict directional stability on box wings. **Water instability in Gazebo returns.** The DART plus single-link plus few-primitives recipe is the mitigation; keep a boat-only world to separate hydrodynamics from flight. **Height sensing over glassy water.** The constant-attitude arrival removes the dependence on the last metre. **Scope creep into planing and scooping.** Explicitly deferred; the runout only has to be stable.

8.5 Decisions we need from the supervisor

Delivery date of the SolidWorks model, for the inertia tensor and outer mould line. Confirmation of the control-surface layout on the front and rear wings, so the mixing matrix is right. Sign-off on the touchdown envelope in Table 3 and on the 90% Monte Carlo pass threshold. Confirmation that the UE5 high-fidelity work is deferred for this phase.

9 In plain terms

We are teaching an unusual double-winged water-plane to fly itself and land gently on water, and we are doing it the careful way. First we write down exactly what a good landing means: how fast, how steep, how softly. Then we build an honest computer model of the aircraft with proper aerodynamics software instead of textbook guesses, and we check that model against known physics before we trust it. Once it is trustworthy we work out the autopilot’s settings with control theory, making sure each loop has healthy safety margins, rather than turning knobs until it looks fine. We then load those settings into PX4, the same open-source autopilot that flies real drones, and fly the aircraft in Gazebo, which can also float it on water. The hard part is landing on calm water, because the aircraft, like a real seaplane pilot, cannot tell exactly how high it is above a mirror-flat surface, so we teach it to hold a steady nose-up attitude and a slow, steady descent until it simply touches. Finally we run about a hundred landings with random wind and weight to prove it is reliable. Along the way the newest member of the team learns the whole toolkit by building real pieces of the project.

10 References and links

Textbooks. R. Beard and T. McLain, *Small Unmanned Aircraft: Theory and Practice*, Princeton University Press, 2012; companion code https://github.com/byu-magicc/mavsim_public. B. Etkin and L. Reid, *Dynamics of Flight: Stability and Control*, 3rd ed., Wiley, 1996. B. Stevens, F. Lewis and E. Johnson, *Aircraft Control and Simulation*, 3rd ed., Wiley, 2015. R. Nelson, *Flight Stability and Automatic Control*, 2nd ed., McGraw-Hill, 1998.

Seaplane practice. FAA-H-8083-23, *Seaplane, Skiplane, and Float/Ski Equipped Helicopter Operations Handbook*, https://www.faa.gov/regulations_policies/handbooks_manuals/aviation/seaplane_handbook. AOPA, “Technique: Glassy water landings”, 2017. D. Savitsky, “Hydrodynamic design of planing hulls”, *Marine Technology* 1(1), 1964.

PrandtlPlane. A. Frediani, V. Cipolla and V. Oliviero, “IDINTOS: the first prototype of an amphibious PrandtlPlane-shaped aircraft”, *Aerotecnica Missili & Spazio* 94, 2015, doi:10.1007/BF03404701. V. Oliviero, D. Zanetti and V. Cipolla, “Flight dynamics model for preliminary design of PrandtlPlane wing configuration with sizing of the control surfaces”, *Aerotecnica Missili & Spazio* 95, 2016, doi:10.1007/BF03404728. “Preliminary analysis of the stability and controllability of a box-wing aircraft configuration”, *Aerospace* 10(10):874, 2023. PARSIFAL project, <https://cordis.europa.eu/project/id/723149>.

Flying qualities. MIL-F-8785C, *Flying Qualities of Piloted Airplanes*, 1980. MIL-STD-1797A.

PX4. Controller diagrams https://docs.px4.io/main/en/flight_stack/controller_diagrams. Fixed-wing rate and attitude tuning https://docs.px4.io/main/en/config_fw/pid_tuning_guide_fixedwing. Altitude and position tuning (TECS, NPF) https://docs.px4.io/main/en/config_fw/position_tuning_guide_fixedwing. Autotune https://docs.px4.io/main/en/config/autotune_fw. Fixed-wing landing https://docs.px4.io/main/en/flight_modes_fw/mission. Land detector https://docs.px4.io/main/en/advanced_config/land_detector. AVL automation tool https://docs.px4.io/main/en/sim_gazebo_gz/tools_avl_automation. Gazebo simulation https://docs.px4.io/main/en/sim_gazebo_gz/. Models <https://github.com/PX4/PX4-gazebo-models>.

Gazebo and water. AdvancedLiftDrag https://gazebo-sim.org/api/sim/8/classgz_1_1sim_1_1systems_1_1AdvancedLiftDrag.html. Buoyancy https://gazebo-sim.org/api/sim/8/classgz_1_1sim_1_1systems_1_1Buoyancy.html. Hydrodynamics and theory https://gazebo-sim.org/api/sim/8/theory_hydrodynamics.html. Create a surface vehicle https://gazebo-sim.org/api/sim/9/surface_vehicles.html. VRX <https://github.com/osrf/vrx>. asv_wave_sim https://github.com/srmainwaring/asv_wave_sim. ArduPilot Gazebo https://github.com/ArduPilot/ardupilot_gazebo. iq_sim https://github.com/Intelligent-Quads/iq_sim.

Aerodynamic tools. AVL <https://web.mit.edu/drela/Public/web/avl/>. XFLR5 <https://www.xflr5.tech/>. OpenVSP and VSPAERO <https://openvsp.org/>.

Direct lift control. NASA CR-165 series, “Development and evaluation of automatic landing control laws for light wing loading STOL aircraft” (Twin Otter DLC autoland), <https://ntrs.nasa.gov/>.